SMOLVLA / LIBERO | 150-EPISODE EXPERIMENT

How often should a robot replan?

Three tasks, ten initial states, five replanning intervals

We vary the number of actions K executed before obtaining a fresh observation and replacing the plan. The study measures task completion, motion, grasping behavior, model calls and runtime for one pretrained model.

OBSERVATION
2 cameras + state + instruction

>

PREDICT
H = 50 actions

>

EXECUTE
First K actions

>

REPLAN
New observation; replace tail

Experimental design

Factor	Setting	Comparison
Tasks	Drawer, soup can, black bowl	Three different manipulation behaviors
Initial states	0-9 for each task	The same ten states for every K
Replanning interval	K = 1, 5, 10, 25, 50	30 episodes per K; 150 in total

All 150 episodes contribute to the main analysis. Each setting has the same task and initial-state coverage; differences reflect the executed prefix length K under the fixed protocol.

Task	Suite	Step limit
Soup can into basket	libero_object	280
Black bowl onto plate	libero_spatial	280
Open the middle drawer	libero_goal	300

Model and controlled conditions

SmolVLA uses flow matching to predict H=50 actions from two images, robot state and a language instruction. Each action specifies relative end-effector translation, rotation and a gripper command. K is an executed prefix length, not repeated execution of one action.

K=1,5,10,25,50; one policy seed (11); 20 Hz simulated control; 10 generation iterations; batch=1; precision fp32; 10 settling steps; 2 warmup calls. Checkpoint, preprocessing, simulator, hardware and logging are held fixed. Physical initial states and initial observations match across K by SHA-256.

Run order is randomized within each source run. Model sampling noise is tied to policy seed and simulation step. The simulator pauses during inference.

Task completion across 150 episodes

3 tasks x 10 initial states x 5 K values. Each K has 30 episodes; each task/K has 10. Success is LIBERO's native signal before the step limit.

K	Drawer	Soup can	Bowl	Total	95% interval
1	9/10	6/10	8/10	23/30	[63.3, 90.0]%
5	10/10	9/10	7/10	26/30	[73.3, 96.7]%
10	10/10	10/10	4/10	24/30	[70.0, 90.0]%
25	1/10	6/10	4/10	11/30	[20.0, 53.3]%
50	3/10	3/10	3/10	9/30	[13.3, 46.7]%

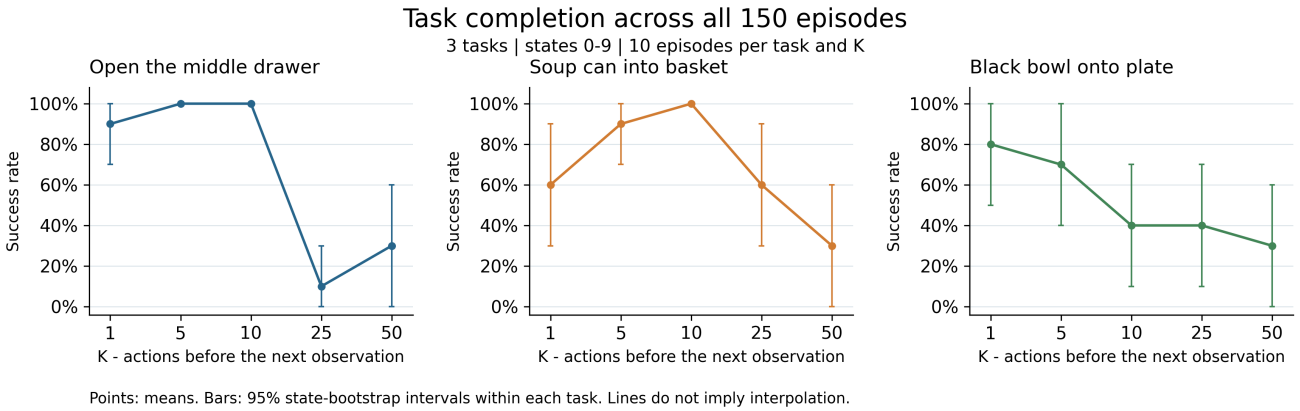


Figure 1. Success by task across all ten initial states. Every K is evaluated from the same states within each task.

Matched contrasts against K=10

K	Success difference, percentage points	95% paired interval
1	-3.3	[-20.0, 13.3]
5	+6.7	[-10.0, 20.0]
10	+0.0	[0.0, 0.0]
25	-43.3	[-60.0, -26.7]
50	-50.0	[-70.0, -30.0]

Means weight tasks equally, then initial states equally. 5,000 bootstrap resamples draw initial-state clusters with replacement within each task; all K outcomes stay paired in every draw. The contrast interval is computed from paired differences, not by subtracting two marginal interval bounds. Tasks and the policy seed remain fixed. Intervals are exploratory and are not adjusted for multiple comparisons.

Which initial situations change outcome?

Each row below holds the task and initial state fixed while K changes. Each cell shows one native LIBERO outcome, not a probability. Reading across a row makes disagreements visible that an aggregate success percentage can hide.

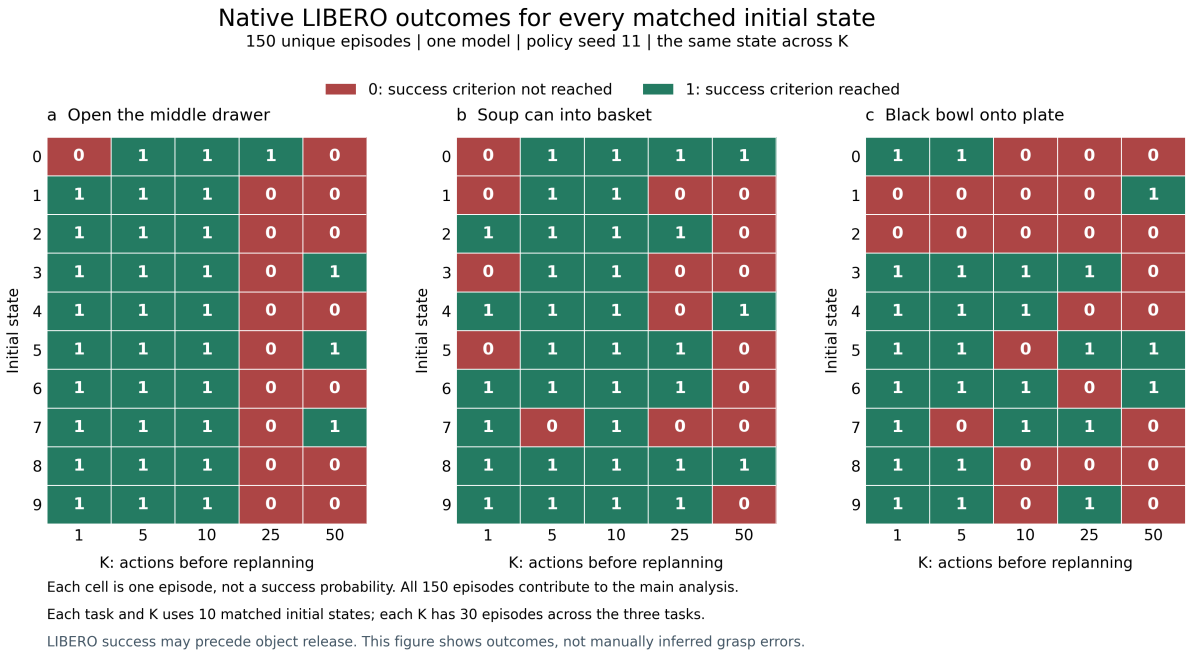


Figure 2. All 150 episode outcomes. Green cells reach the simulator's success condition; red cells reach the step limit without success.

Paired outcomes compared with K=10

K	K succeeds; reference fails	K fails; reference succeeds	Same outcome
1	4	5	21
5	4	2	24
10	0	0	30
25	2	15	13
50	2	17	11

No interval succeeds in every initial situation. The result depends on the task and initial geometry; different settings can have similar totals while failing in different scenes. The paired intervals on page 2 quantify uncertainty across these states, conditional on the fixed tasks and policy seed.

The examples on page 6 examine visible behavior in selected recordings. They illustrate particular failures and recoveries without replacing the complete outcome grid.

Computational cost and robot motion

Runtime values below are means per episode. Early termination affects calls, duration and path length; calls = ceil(executed steps / K). A low cost is not a substitute for successful completion.

K	Calls	Model, s	Simulator, s	Sum, s	Episode, s
1	162.0	125.2	3.6	128.8	132.5
5	31.3	24.3	3.4	27.7	31.7
10	15.5	12.0	3.5	15.5	19.0
25	9.6	7.5	5.0	12.5	16.9
50	5.1	4.1	5.2	9.3	14.0

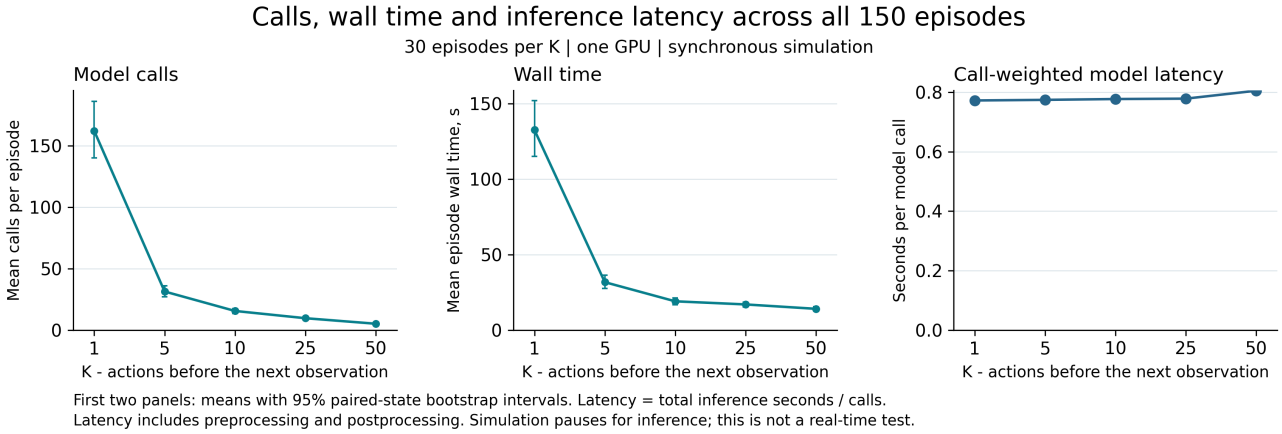


Figure 3. Calls and wall time across all 150 episodes, with state-bootstrap intervals; model latency is total inference time divided by total calls, without an uncertainty interval.

Motion proxies

K	EEF path, m	Plan revision	Replan jump	Gripper closes
1	0.88	0.317	0.337	9.4
5	0.98	0.320	0.334	4.3
10	1.06	0.417	0.440	5.6
25	1.38	0.527	0.534	4.6
50	1.29	undefined	0.455	3.4

Plan revision: L2 difference between the new first translation command and the next command in the previous plan. At K=50 the previous plan is exhausted, so revision is undefined. Replan jump: L2 change between consecutive executed translation commands at replanning. Both use action units, not meters. Changes and gripper-closing transitions are proxies; they do not count grasp errors or prove useful correction. Each proxy mean uses only episodes where that value is defined.

Model time includes preprocessing, inference, postprocessing and GPU synchronization. Simulator time includes stepping and rendering. Episode wall time adds reset, frame recording and video finalization, but excludes model loading, warmup and JSON-log writes. Simulated duration is steps / 20, not wall time.

Measured totals: 6,707 model calls and 107.1 min summed episode time across all 150 episodes. Setup and transfer time are excluded.

Visual annotation: coverage and errors

The review subset contains 60 episodes: every task and K for states 0-3. 60 recordings were inspected; both missed-grasp and drop counts are resolvable in 41. States 4-9 contribute quantitative outcomes and cost but are outside this event-annotation subset. Unresolved contacts remain unknown.

All episodes	Selected	Inspected	Both counts known
150	60	60	41

Review coverage and observed events by K

K	Selected / all	Inspected	Both known	Misses: sum [n/all]	Drops: sum [n/all]
1	12/30	12	7	0 [7/30]	0 [7/30]
5	12/30	12	10	0 [10/30]	0 [10/30]
10	12/30	12	9	0 [9/30]	0 [9/30]
25	12/30	12	8	0 [8/30]	2 [9/30]
50	12/30	12	7	1 [7/30]	0 [7/30]

Inspected counts come only from inspection.json. If no inspection record is supplied, the report says unknown; it does not infer inspection from a CSV row. reviewed=yes denotes both event counts being resolvable. reviewed=no can mean an inspected but ambiguous episode. Each event has its own known denominator: a drop may be clear while missed grasps remain unknown. Unknown events are never zero.

Correction labels in the selected subset

Selected episodes	Yes	No	Unclear	Unavailable
60	11	11	38	0

Review method and interpretation

The assistant reviews both camera views using sampled frame sheets, transitions of the gripper command, the final action and outcome frames; ambiguous contacts receive denser frame checks where available. Exact reviewed steps and evidence are recorded in notes and inspection metadata. There is no independent human review. Counts are observations from this subset, not estimated rates for all 150 episodes.

A missed grasp means closing without retaining the target. Failure to hook a drawer handle with an open gripper is not a missed finger grasp under this definition. A drop requires unintended loss after retention; planned release during placement is excluded. A correction label requires visible redirection plus relevant replanning steps in logs; temporal alignment does not establish a causal benefit.

Illustrative cases: behavior and errors

These paired examples use the same initial situation within each task. Outcomes below come from episode summaries; the accompanying observations are grounded in reviewed videos and logs.

1. A lost can is not recovered

Soup can into basket; state 3; seed 11. K=10: success, 139 steps, 14 calls; K=25: failure, 280 steps, 12 calls.

Observation. K=10 retains the can during transport and reaches native success at step 139. K=25 lifts it, then loses it around steps 82-86 outside the basket. The hand continues toward the basket after replans at 100 and 125; it does not retrieve the can before the step-280 timeout.

Hypothesis. Slippage or contact geometry may explain the loss. A longer interval may leave fewer opportunities to adjust the grasp, but the recordings do not isolate the cause of the drop.

Evidence. Object/state3/seed11; both videos and review-object.json. Dense frames 76-94; gripper commands stay positive (0.987-1.000), so the loss is not a commanded release.

2. A second approach still misses placement

Black bowl onto plate; state 3; seed 11. K=1: success, 80 steps, 80 calls; K=50: failure, 280 steps, 6 calls; K=10: success, 75 steps, 8 calls.

Observation. K=50 withdraws at steps 80-100, returns at 105-125 after replan 100, transports the bowl at 130-160 and opens near the plate rim around 160. Placement remains incomplete at 280. K=1 succeeds at 80 using 80 calls; K=10 also succeeds at 75 using only 8 calls.

Hypothesis. A new plan can redirect an approach without ensuring accurate placement. The K=1 versus K=10 contrast shows that extra updates did not improve this binary outcome; it does not establish general equivalence.

Evidence. Spatial/state3/seed11; videos, steps/chunks and review-bowl.json. Replan 100 command jump 0.742472. Initial K=50 contact counts remain unknown.

3. A longer plan opens the drawer sooner

Open the middle drawer; state 3; seed 11. K=5: success, 252 steps, 51 calls; K=50: success, 127 steps, 3 calls.

Observation. Both episodes succeed. At K=5, the hand remains near the handle around steps 90-215 with little opening, then finishes at 252 after 51 calls. K=50 visibly pulls around 100-127 and succeeds at 127 after 3 calls. Measured compute time is 44.7 versus 4.9 seconds.

Hypothesis. A longer action sequence may sustain an effective pull in this state. Contact forces are not logged, so the delay at K=5 cannot be attributed uniquely to replanning.

Evidence. Goal/state3/seed11; matched initial-state and observation hashes; both videos, summary logs and review-drawer.json. Open-finger hooking is not a missed closing grasp.

Examples illustrate mechanisms and uncertainty; they do not replace the complete outcome table. A REPLAN marker shows plan replacement, not proof that a new image caused an improvement.

Findings and limits of inference

Findings from the complete experiment

1. Across all 150 episodes, K=1, 5, 10, 25 and 50 succeed in 23/30, 26/30, 24/30, 11/30 and 9/30 matched situations. K=5 has the highest observed success count, while its 27.7 s mean compute is 4.7 times lower than the 128.8 s at K=1.

2. Relative to K=10, paired success differences are -3.3 percentage points for K=1 and +6.7 for K=5, with exploratory 95% intervals [-20.0, 13.3] and [-10.0, 20.0]. These intervals include zero: the shorter intervals cannot be reliably ranked or declared equivalent. K=10 uses 15.5 s mean compute.

3. K=25 and K=50 have lower aggregate success. Their paired differences from K=10 are -43.3 and -50.0 percentage points, with exploratory 95% intervals [-60.0, -26.7] and [-70.0, -30.0]. Individual outcomes remain task- and state-dependent, so longer intervals can still succeed where another setting fails.

4. Reviewed videos show an unrecovered can drop, a repeated bowl approach that misses placement, and a successful drawer pull with fewer calls at K=50. Two state-2 drawer failures open the upper drawer instead of the middle one. Visible redirection after replanning does not guarantee recovery; contact-mechanism explanations remain hypotheses.

What the comparison identifies

Changing K changes both the observation interval and how often stochastic action chunks are regenerated. The comparison identifies the effect of the execution policy K; it cannot isolate fresh visual feedback from plan replacement or stochastic regeneration. Matching sampling noise at shared simulation steps does not create common subsequent trajectories.

Remaining limitations

The experiment covers ten states per task, but only three selected tasks, one checkpoint and one policy seed. Bootstrap intervals describe variability across these states conditional on those choices. They do not estimate variation across new tasks, models, seeds or physical robots. The multiple K comparisons are exploratory. These simulator states are not claimed to be unseen during checkpoint training.

Native LIBERO success can occur while an object is still held in the target region. Episodes stop at that signal; stable placement after explicit release was not tested separately. Early termination also changes cost and path length. Visual event counts cover a subset and retain unresolved contacts as unknown.

Physics pauses while inference runs. A 20 Hz simulated controller is not evidence of real-time operation under model latency. Sessions can also differ in system load. No conclusion about a physical robot is supported by these simulations alone.

Conclusion

Replanning frequency changes computational cost and task behavior. K=5 and K=10 offer useful observed cost-success compromises in this setup. Updating after every action increases compute without exceeding the success count at K=5. Longer intervals reduce model calls but lose aggregate success, with exceptions in individual situations. No universal optimum follows from three tasks, ten initial states per task and one policy seed. Native success, assistant-only event annotation and synchronous simulation limit claims about stable placement, causal correction and real-time robots.

Next controlled tests

Multiple policy seeds and additional tasks would test robustness. A separate intervention comparing fresh and stale observations at the same regeneration schedule would help separate feedback from chunk replacement. Adaptive K and physical-robot latency require their own evaluation protocols.

Reproducibility and source provenance

Device: NVIDIA GeForce RTX 2060; CUDA 12.6; Python 3.12.3; Linux. LeRobot 0.6.1; hf-libero 0.1.4; MuJoCo 3.3.2; PyTorch 2.7.1.

Model: HuggingFaceVLA/smolvla_libero
Revision: 6721902bc4d61e50a3bfdb11dfb4cb626f05d102
Backbone: HuggingFaceTB/SmolVLM2-500M-Instruct
Revision: 7b375e1b73b11138ff12fe22c8f2822d8fe03467

Rebuild results from the repository root

```
python scripts/analyze_extension.py
python scripts/build_expanded_figures.py --overwrite
python scripts/build_state_outcomes.py --overwrite
python scripts/build_expanded_report.py --overwrite
python scripts/build_expanded_demo.py --overwrite
```

See docs/linux-checklist.md for collecting the episodes and setting up the pinned runtime.

The analysis requires complete paired episodes and matching runtime, checkpoint and frozen core code. Finish visual review and save annotations.csv plus inspection.json before rebuilding statistics. Then author discussion.json from actual videos. Plot regeneration after an edit uses --overwrite. PDF generation needs ReportLab and pypdf, followed by page rendering and visual inspection.

Frozen core SHA-256: d96cfeded0dc6e1ca630d2fbdc9f23e1cdf51806bfbe14211327200da412902e
Dependency lock SHA-256: 8574b51f0fad0b6ec40c02b3c6192da4606e30a94fd8dc1e18068b461d81212a
Expanded statistics SHA-256: 1d3d77bdb4bb85fc5772bc7031fc601a7e432f43655ee612f4511d9be6f2e576
Study discussion SHA-256: 0d93f0cd2163965ee495ec506c133395b44296729b5729a0c4a3f839981e3d5b

The statistics record every source manifest hash and every raw summary, action log, chunk log and video hash. Original episode files and their fingerprints remain unchanged. PDF provenance binds the statistics, discussion and figures to this report through SHA-256 digests.

References

1. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics
2. Evaluated SmolVLA LIBERO checkpoint (pinned revision)
3. LeRobot 0.6.1 SmolVLA implementation
4. LeRobot 0.6.1 LIBERO adapter
5. LIBERO environment documentation

References describe the model and software. All experimental numbers come from the actual source episodes; interpretation is supplied in the reviewed discussion files.

SMOLVLA / LIBERO | MOVING-TARGET STRESS TEST

Replanning while the target moves

The soup-can task is modified by a controlled external force before finger contact. The question is whether sensitivity to this intervention changes with K. These 135 episodes have their own matched static control and are not pooled with the standard-task results.

Factor	Fixed setting
Task and horizon	Alphabet soup can into basket; 280 actions at 20 Hz
Matched design	States 10-14; policy seeds 11, 22, 33; K = 5, 10, 50
Conditions and size	Static, slow reference, fast reference; 15 episodes per cell; 135 total
Policy and order	Same pinned SmolVLA, H=50, FP32; condition/K order randomized within state/seed

A bounded point force, released at first finger contact

The x reference has amplitude 2 cm and periods 8/4 s. It starts at 0.5 simulated seconds with a 1 s smoothstep ramp. A PD controller uses kp=100 N/m, kd=0.5 N s/m and a 1 N cap. Its application point is 3 cm below the center of mass; torque_y = offset_z * force_x. The can is never teleported and there is no separate orientation servo.

The first finger/pad contact permanently removes force and moment. Contact is not a verified grasp. Static uses the same instrumentation with zero intervention force. The intervention never triggers an extra model call; simulation pauses during inference.

Physical calibration with the robot stationary

Physical diagnostic	Static	Slow	Fast
Actual x range (mm)	0.00	35.06	35.14
Actual y range (mm)	0.00	4.37	2.79
Actual z range (mm)	0.000	0.246	0.369
Maximum axis tilt from initial (degrees)	0.00	8.58	11.49
Active mean tracking error (mm)	n/a	2.21	2.23
Peak applied force (N)	0.000	0.275	0.291
Enabled samples at force cap (%)	n/a	0.0	0.0

Calibration: excluded state 20, 240 actions (12 simulated seconds), no VLA inference, zero robot motion and open gripper. The selected controller still permits tilt and lateral drift. Slow/fast label reference periods, not perfectly isolated measured object speeds. Parameters were frozen from physical tracking and stability before collecting the main outcomes.

Completion and computational cost

Every planned episode contributes, including no-contact episodes and unsuccessful transfers. Each condition/K cell contains five initial states with three policy-seed repeats.



Vertical intervals resample initial states with their complete seed/treatment blocks. They are exploratory 95% percentile intervals, not independent-episode intervals.

Condition	K	Success	Mean calls	Compute (s)	Wall time (s)
Static	5	12/15 (80.0%)	35.8	31.98	36.70
Static	10	13/15 (86.7%)	17.3	17.18	21.79
Static	50	6/15 (40.0%)	4.8	8.17	13.59
Slow	5	8/15 (53.3%)	42.1	37.57	42.74
Slow	10	13/15 (86.7%)	17.4	17.42	22.04
Slow	50	5/15 (33.3%)	5.1	8.69	14.19
Fast	5	13/15 (86.7%)	32.9	29.48	34.00
Fast	10	15/15 (100.0%)	14.6	14.69	18.93
Fast	50	6/15 (40.0%)	4.8	8.18	13.55

Static-control successes by seed 11/22/33: K=5: 5/5/2; K=10: 5/4/4; K=50: 1/0/5. Each count is out of five initial states. These seed comparisons are descriptive; the uncertainty analysis remains conditional on the three selected seeds.

Timing definitions

Compute time is synchronized inference plus simulator stepping. Episode wall time also includes reset, in-memory telemetry and video creation, recording and finalization; it excludes JSON log writing, checksums, model loading and warmup. It is not a real-time robot latency benchmark: the simulated target does not keep moving while the model is computing.

Interpret the cost and success columns together. Success can terminate an episode early, so a lower average cost can result from fewer calls per action, a shorter episode, or both. The fixed 280-action time limit is shared across conditions.

Across the full stress test: 91/135 successes, 2,622 model calls, 25,703 executed actions and 54.4 minutes of accumulated episode wall time. This total is descriptive and is not an aggregate benchmark score.

Does target motion change the effect of K?

First compare each moving condition with its own static control at the same K. The difference is moving minus static success, expressed in percentage points (pp). A negative value indicates lower completion under the force intervention.

Reference motion	K	Paired change (pp)	Exploratory 95% interval
Slow	5	-26.7	[-46.7, -6.7]
Slow	10	+0.0	[-20.0, 20.0]
Slow	50	-6.7	[-20.0, 0.0]
Fast	5	+6.7	[0.0, 20.0]
Fast	10	+13.3	[0.0, 26.7]
Fast	50	+0.0	[0.0, 0.0]

Interaction relative to K=10

Interaction = (moving - static at K) - (moving - static at K=10). A positive value means the motion-related change is more favorable at that K than at K=10. This contrast accounts for baseline performance differences while preserving paired starts.

Reference motion	K vs. 10	Interaction (pp)	Exploratory 95% interval
Slow	5	-26.7	[-46.7, -6.7]
Slow	50	-6.7	[-26.7, 13.3]
Fast	5	-6.7	[-26.7, 13.3]
Fast	50	-13.3	[-26.7, 0.0]

What the uncertainty calculation does

The 5,000-draw bootstrap resamples five initial states. Every sampled state retains all three policy seeds, conditions and K values. Seed repeats are averaged within a state; they are not treated as additional independent scenes. All contrasts share the same state draws (analysis seed 20261024).

With only five state units, uncertainty estimates are weak and can be degenerate for identical outcomes. Intervals are conditional on the chosen seed set, are not adjusted for multiple comparisons and do not establish equivalence when they include zero. The interaction also cannot remove success floor/ceiling constraints or the dependence of exposure duration on contact.

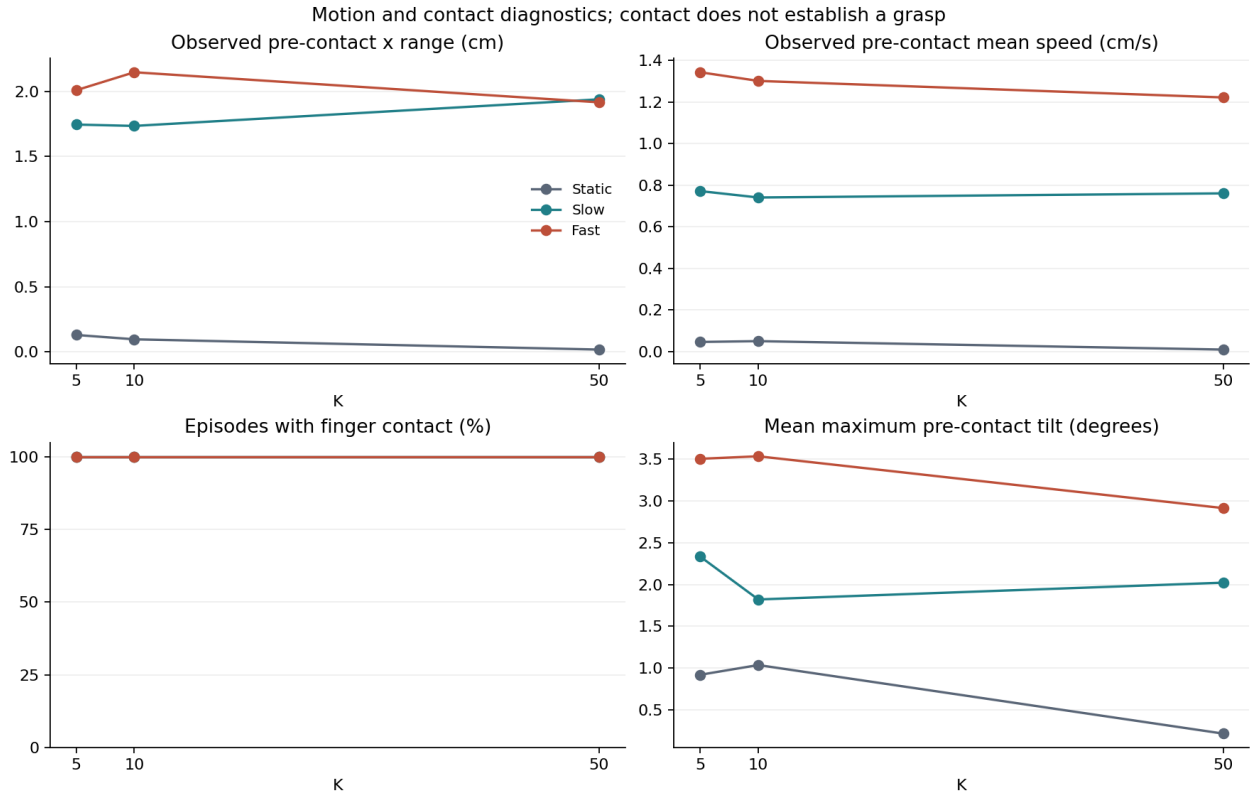
Fixed reference phase also matters

With the 0.5 s onset, the first scheduled replan strictly after onset is at 0.75, 1 and 2.5 s for K=5, 10 and 50, if the episode continues. At 2.5 s, the fast reference has returned to its initial x position while the slow reference is near its maximum offset. The real object may lag. An oscillating target can therefore return toward an older planned grasp position: a shorter reference period need not be harder. This is a possible timing mechanism, not a verified explanation of an individual outcome.

Raw cell counts appear on the preceding page. Per-episode state/seed outcomes and all four cell counts used in each interaction are retained in the CSV exports.

Observed motion and contact

The commanded reference does not guarantee an identical object trajectory. These measurements check motion actually achieved before the first finger contact.



Ranges and tilt are measured at physics-substep resolution. Mean speed is a control-rate finite-difference diagnostic. Exposure ends at first contact or episode termination.

Condition	K	Finger contact	No contact	Mean contact time (s)*
Static	5	15/15	0	2.57
Static	10	15/15	0	2.62
Static	50	15/15	0	2.46
Slow	5	15/15	0	2.56
Slow	10	15/15	0	2.55
Slow	50	15/15	0	2.83
Fast	5	15/15	0	2.61
Fast	10	15/15	0	2.57
Fast	50	15/15	0	2.42

*Conditional on an observed contact; no-contact episodes are excluded from this time mean and counted explicitly. A single finger brush can stop the force without securing the can. Contact is not a verified grasp, recovery event or successful placement.

All completed episodes passed the force-cap, fixed-K schedule and permanent force/moment release checks. Tracking error, force saturation, x/y/z ranges and exposure are retained in the raw telemetry and analysis exports.

Interpretation and research contribution

Measured changes in completion

Slow reference: -26.7 pp at K=5; +0.0 pp at K=10; -6.7 pp at K=50.

Fast reference: +6.7 pp at K=5; +13.3 pp at K=10; +0.0 pp at K=50.

The exploratory interval contains zero for 3 of 4 condition-by-K interaction contrasts. These results describe sensitivity to this particular force rule; they do not establish a universal optimal K or an isolated effect of object speed.

Contribution in the context of action chunking

ACT (Zhao et al., 2023) predicts action sequences and combines overlapping predictions through temporal ensembling. Its chunk-length ablation trained separate policies for each length. Here the checkpoint and H=50 remain fixed while the executed prefix K changes [1].

RTC (Black et al., 2025) generates the next chunk during execution, retaining actions already committed. Our simulator pauses during inference; this is a synchronous replanning study, not an evaluation of asynchronous latency robustness [2].

The empirical contribution is a controlled comparison of execution intervals, coupled with a calibrated force intervention, matched starts, nested policy-seed repeats and paired condition-by-K contrasts. It studies one pretrained policy rather than introducing a new action-chunking algorithm.

Preselected visual illustration

Assistant review of state 10/seed 11 shows fast K=50 contacting the can at 2.112 s and commanding closure at step 45, then transporting an empty gripper toward the basket without retrieval through 280 actions despite later replans. Fast K=10 returns after its first unsuccessful closure and succeeds in 153 actions/16 calls; fast K=5 succeeds in 141 actions/29 calls. This single illustration gives neither error rates nor a causal explanation.

Validation and limits

All 135 logs matched saved predictions and fixed-K schedules; all 135 videos were decoded. Across 45 state/seed/K triplets (90 moving-versus-static comparisons), pre-motion executed actions and full predictions were exactly identical: maximum differences were zero.

These are instrumentation checks, not independent proof of physical success. Contact is not a grasp label; sampled assistant review leaves grasp/drop counts unresolved. Contact release changes exposure, and native success does not establish stable placement.

References and reproducible evidence

1. Zhao et al. (2023). Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. RSS.

2. Black, Galliker and Levine (2025). Real-Time Execution of Action Chunking Flow Policies. arXiv:2506.07339.

Protocol: docs/moving-target-protocol.md. Statistics, action/video verification, sampled visual review and full provenance accompany the report. Rebuild with analyze_moving_target.py, verify_moving_target.py --decode-video and build_moving_target_report.py.